

HDOC Day-34

Verification of Distributive Properties of Matrix Operations Using C

Aim

To write a menu-driven C program using separate functions and variable-length 2D arrays to verify the left and right distributive properties of matrix multiplication over matrix addition. The program displays both the left-hand-side and right-hand-side result matrices and states whether the selected property is verified.

Algorithm

1. Start the program.
2. Display a menu: (1) Left distributive property, (2) Right distributive property, (3) Exit.
3. Accept the user choice and use a switch statement.
4. For the left distributive property, read A ($m \times n$), B ($n \times p$), and C ($n \times p$). Compute $B+C$, then $A*(B+C)$. Separately compute $A*B$ and $A*C$, then add them. Compare the two $m \times p$ matrices.
5. For the right distributive property, read A ($m \times n$), B ($m \times n$), and C ($n \times p$). Compute $A+B$, then $(A+B)*C$. Separately compute $A*C$ and $B*C$, then add them. Compare the two $m \times p$ matrices.
6. Display the LHS matrix, RHS matrix, and whether the property is verified.
7. Repeat until Exit is selected. Display an error for an invalid menu choice.
8. Stop.

C Program

```

#include <stdio.h>

void readMatrix(int r, int c, int a[r][c]);
void printMatrix(int r, int c, int a[r][c]);
void addMatrix(int r, int c, int a[r][c], int b[r][c], int s[r][c]);
void multiplyMatrix(int r1, int c1, int c2,
                    int a[r1][c1], int b[c1][c2], int p[r1][c2]);
int equalMatrix(int r, int c, int a[r][c], int b[r][c]);
void leftDistributive(void);
void rightDistributive(void);

int main(void)
{
    int choice;
    do {
        printf("\n----- MATRIX DISTRIBUTIVE PROPERTY MENU -----\n");
        printf("1. Verify Left Distributive Property\n");
        printf("2. Verify Right Distributive Property\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: leftDistributive(); break;
            case 2: rightDistributive(); break;
            case 3: printf("Exiting program...\n"); break;
            default: printf("Invalid choice! Please enter 1 to 3.\n");
        }
    } while (choice != 3);
    return 0;
}

void readMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void printMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void addMatrix(int r, int c, int a[r][c], int b[r][c], int s[r][c])
{
    int i, j;

```

```

        for (i = 0; i < r; i++)
            for (j = 0; j < c; j++)
                s[i][j] = a[i][j] + b[i][j];
    }

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1], int b[c1][c2], int p[r1][c2])
{
    int i, j, k;
    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            p[i][j] = 0;
            for (k = 0; k < c1; k++) p[i][j] += a[i][k] * b[k][j];
        }
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

void leftDistributive(void)
{
    int m, n, p;
    printf("Enter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);
    int A[m][n], B[n][p], C[n][p];
    int BC[n][p], AB[m][p], AC[m][p], lhs[m][p], rhs[m][p];

    printf("Enter matrix A (%d x %d):\n", m, n); readMatrix(m,n,A);
    printf("Enter matrix B (%d x %d):\n", n, p); readMatrix(n,p,B);
    printf("Enter matrix C (%d x %d):\n", n, p); readMatrix(n,p,C);

    addMatrix(n,p,B,C,BC);
    multiplyMatrix(m,n,p,A,BC,lhs);
    multiplyMatrix(m,n,p,A,B,AB);
    multiplyMatrix(m,n,p,A,C,AC);
    addMatrix(m,p,AB,AC,rhs);

    printf("LHS = A*(B+C):\n"); printMatrix(m,p,lhs);
    printf("RHS = (A*B) + (A*C):\n"); printMatrix(m,p,rhs);
    printf(equalMatrix(m,p,lhs,rhs) ? "Left distributive property is verified.\n"
                                     : "Left distributive property is not verified.\n");
}

void rightDistributive(void)
{
    int m, n, p;

```

```

printf("Enter m, n and p: ");
scanf("%d %d %d", &m, &n, &p);
int A[m][n], B[m][n], C[n][p];
int AB[m][n], AC[m][p], BC[m][p], lhs[m][p], rhs[m][p];

printf("Enter matrix A (%d x %d):\n", m, n); readMatrix(m,n,A);
printf("Enter matrix B (%d x %d):\n", m, n); readMatrix(m,n,B);
printf("Enter matrix C (%d x %d):\n", n, p); readMatrix(n,p,C);

addMatrix(m,n,A,B,AB);
multiplyMatrix(m,n,p,AB,C,lhs);
multiplyMatrix(m,n,p,A,C,AC);
multiplyMatrix(m,n,p,B,C,BC);
addMatrix(m,p,AC,BC,rhs);

printf("LHS = (A+B)*C:\n"); printMatrix(m,p,lhs);
printf("RHS = (A*C) + (B*C):\n"); printMatrix(m,p,rhs);
printf(equalMatrix(m,p,lhs,rhs) ? "Right distributive property is verified.\n"
                                : "Right distributive property is not verified.\n");
}

```

Test Case Table

Test	Choice	Dimensions	Expected Result
1	1	m=2, n=2, p=2	LHS and RHS are equal; left distributive property verified
2	2	m=2, n=2, p=2	LHS and RHS are equal; right distributive property verified
3	1	m=2, n=3, p=2	Both result matrices have dimension 2 x 2 and are equal
4	2	m=3, n=2, p=2	Both result matrices have dimension 3 x 2 and are equal
5	4	-	Invalid choice message
6	3	-	Program exits

Result

The menu-driven C program was successfully designed using separate functions and variable-length 2D arrays. It verifies the left distributive property $A*(B+C) = (A*B) + (A*C)$ and the right distributive property $(A+B)*C = (A*C) + (B*C)$. For each operation, the program displays both $m \times p$ result matrices and states whether the selected distributive property is verified.